

Guía esencial de plataformas para APIs: monitoreo, logs, trazas y seguridad

FastAPI (microservicios y APIs)

Framework asincrónico para construir APIs HTTP tipadas y de alto rendimiento.

Qué resuelve

Exponer servicios HTTP/JSON con validación automática de datos, documentación OpenAPI, y soporte nativo para asincronía (uvicorn + Starlette).

Conceptos clave

- **Rutas y controladores:** funciones Python decoradas que atienden métodos HTTP.
- **Modelos Pydantic:** validan y tipan entradas/salidas.
- **Dependencias:** inyectan lógica transversal (auth, DB, rate limit).
- **OpenAPI/Swagger:** documentación y pruebas desde el navegador.

Buenas prácticas

- Versionar rutas (`/v1/...`), validar todo, manejar errores con excepciones controladas, y devolver códigos HTTP precisos (200/201/204/400/401/403/404/409/422/429/5xx).
- Añadir *headers de seguridad* (nosniff, X-Frame-Options, COOP/CORP), *no-cache* para respuestas sensibles y *X-Correlation-Id* para trazabilidad.
- Exponer métricas y health checks; separar lectura/escritura; paginar; usar idempotencia en operaciones con *retries*.

Prometheus

Base de series temporales para recopilar métricas por “pull” y consultarlas con PromQL.

Qué resuelve

Recopila métricas numéricas de sistemas (counters, gauges, histograms, summaries), permitiendo analizar tasas, latencias y utilización.

Conceptos clave

- **Modelo de “scrape”**: Prometheus llama a `/metrics` de cada servicio.
- **Tipos**:
 - *Counter*: solo sube (p.ej. peticiones totales).
 - *Gauge*: sube/baja (p.ej. memoria actual).
 - *Histogram/Summary*: distribuciones y percentiles.
- **Labels**: metadatos (p.ej. `status`, `handler`, `method`) que segmentan las métricas.
- **PromQL**: lenguaje para consultas (tasa, agregación, percentiles).

Buenas prácticas

- Mantener baja la **cardinalidad** de labels; definir **nombres consistentes**; usar **recording rules** para consultas pesadas; centralizar **alertas** desde métricas clave.

Grafana

Visualización, paneles y alertas basadas en datos de múltiples fuentes.

Qué resuelve

Construir dashboards operativos y de negocio sobre Prometheus, Elasticsearch u otras fuentes, con filtros, variables y alertas.

Conceptos clave

- **Datasource**: conexión a la fuente (Prometheus, ES, etc.).
- **Paneles**: gráficas, tablas, *stat cards*, *time series*.
- **Variables**: filtrar por servicio, endpoint, entorno, etc.
- **Alerting**: reglas por panel/consulta con umbrales y silencios.

Buenas prácticas

- Paneles por **dominio/servicio**; no mezclar demasiadas métricas en una sola vista; usar **templating** y **provisioning** (JSON) para versionar; definir **roles** y **folders**.

Logstash

Ingesta y transformación de eventos antes de enviarlos a un almacenamiento.

Qué resuelve

Recibir logs en múltiples protocolos (UDP/TCP/HTTP), normalizarlos (parseo, enriquecimiento, anonimización) y redirigirlos a destinos (Elasticsearch, S3, etc.).

Conceptos clave

- **Pipeline:** `input` → `filter` → `output`.
- **Codecs y filtros:** `json`, `grok`, `mutate`, `date`, `geoip`.
- **Backpressure y persistencia:** considerar cola/confirmaciones cuando la confiabilidad importa (TCP/HTTP vs UDP).
- **Enriquecimiento:** añadir campos con servicio, versión, *trace id*, usuario, etc.

Buenas prácticas

- Definir un **esquema de log** consistente (timestamp ISO, nivel, servicio, *cid*, ruta, latencia).
- **Redactar PII** (nombres, correos, tokens).
- Controlar el **volumen** (sampling, niveles de log) y validar que el pipeline no sea cuello de botella.

Elasticsearch

Motor distribuido para indexar, buscar y agregar documentos JSON a gran escala.

Qué resuelve

Búsquedas rápidas y agregaciones sobre logs, métricas de eventos o datos de aplicación.

Conceptos clave

- **Índices y mapeos:** definen cómo se almacenan campos (texto vs numéricos vs keyword).
- **Shards y réplicas:** distribución de datos y tolerancia a fallos.
- **Agregaciones:** histogramas, *terms*, percentiles.
- **ILM (Index Lifecycle Management):** rotación y retención por costos.

Buenas prácticas

- Ajustar **mappings** para latencias como **number**, IDs como **keyword**, mensajes como **text**.
- Evitar explosión de índices; usar **plantillas**; diseñar **nombres de índice** por fecha/entorno.
- Observar consumo de CPU/RAM/heap y *refresh intervals* según la carga.

Kibana

Interfaz de análisis y visualización de datos almacenados en Elasticsearch.

Qué resuelve

Explorar logs en tiempo real, construir visualizaciones y paneles sobre índices, y compartir hallazgos con el equipo.

Conceptos clave

- **Discover:** búsqueda interactiva con filtros, *time picker* y columnas.
- **Data Views** (index patterns): definen qué campos están disponibles.
- **Visualizaciones:** Lens, TSVB, Maps; *dashboards* combinados.
- **Alertas** (si están habilitadas): reglas basadas en consultas.

Buenas prácticas

- Crear **búsquedas guardadas** (p.ej. “5xx últimos 15m”).
- Destacar campos clave (estado HTTP, latencia, *cid*).
- Usar **roles** y **spaces** para segmentar acceso.

Jaeger

Sistema de trazas distribuidas para entender la latencia y el flujo de llamadas entre servicios.

Qué resuelve

Responder “qué pasó durante una solicitud”: qué componentes participaron, cuánto tardaron y dónde se producen cuellos de botella.

Conceptos clave

- **Trace y span**: una traza agrupa spans; cada span representa una operación (con inicio/fin, tags y logs).
- **Propagación de contexto**: IDs que viajan entre servicios (W3C Trace Context, B3).
- **Muestreo (sampling)**: porcentaje de solicitudes trazadas para controlar costos.
- **Etiquetas (tags)**: metadata útil (endpoint, código, tamaño).

Buenas prácticas

- Instrumentar **puntos de latencia** (DB, red, terceros).
- Propagar **trace id** y correlacionarlo con logs/metricas.
- Definir **nombres de operaciones** claros y estables.

OWASP ZAP (Zed Attack Proxy)

Proxy y escáner de seguridad web para pruebas manuales y automatizadas.

Qué resuelve

Detectar problemas comunes de seguridad en aplicaciones web: cabeceras faltantes, recursos cacheables, XSS reflejado, *CSRF* ausente, etc.

Conceptos clave

- **Passive vs Active Scan:** pasivo observa respuestas; activo intenta explotar.
- **Spider/AJAX Spider:** descubre rutas navegables.
- **Contexts:** definen el ámbito, autenticación y reglas de un objetivo.
- **Automation Framework:** planes YAML reproducibles con reportes HTML/XML.

Buenas prácticas

- Limitar el **alcance** del escaneo, sobre todo en entornos compartidos.
- Ajustar **políticas** para reducir falsos positivos y tiempos excesivos.
- Integrar reportes a flujos de QA/seguridad con *evidencias* y severidades.

Endpoints típicos de un microservicio HTTP

Diseño por responsabilidad: salud, observabilidad, recursos y administración.

Salud y disponibilidad

- **GET /health** (*liveness*): responde si el proceso está vivo; usado por orquestadores, balanceadores, pipelines y otros servicios para verificar si la app contesta. Debe ser **rápido** y **barato** (sin depender de terceros).
- **GET /ready** (*readiness*): confirma que dependencias críticas están listas (DB, colas, configuración). Solo cuando es **ready** el orquestador empieza a enviar tráfico.
- **GET /version**: expone versión *semver*, *commit* o *build date* para verificar despliegues.

Observabilidad

- **GET /metrics**: punto de exposición de métricas (Prometheus); incluye contadores, histogramas de latencia, etc.
- **GET /logs/test** (opcional): ruta para validar envío de logs en entornos de prueba.

Recursos protegidos y autorización

- **GET/POST/PUT/DELETE /{recurso}**: operaciones CRUD con autenticación y autorización (RBAC/ABAC).
- **/private/***: rutas que requieren *token* (p.ej., JWT).
 - *401 Unauthorized*: no autenticado o token inválido.
 - *403 Forbidden*: autenticado pero sin permisos.
- **/admin/***: operaciones restringidas a roles elevados; registrar auditorías.

Buenas prácticas transversales

- **Headers de seguridad:** `X-Content-Type-Options: nosniff`, `X-Frame-Options: DENY`, `Referrer-Policy`, políticas COOP/CORP/COEP cuando apliquen.
- **Cache-Control:** `no-store` para respuestas sensibles; definir TTLs explícitos para contenido cacheable.
- **Idempotencia:** claves idempotentes en PUT/POST que puedan repetirse por reintentos.
- **Paginación y filtros:** evitar respuestas enormes; límites por defecto razonables.
- **Correlación:** generar o respetar `X-Correlation-Id` para unir métricas, logs y trazas.
- **Gestión de errores:** payloads de error estructurados (código, mensaje, detalle, *trace id*), sin filtrar información sensible.